PROJECT REPORT

ON

Real-Time Driver's Drowsiness Detection System

(AI Driver Drowsiness Monitoring System using OpenCV and MediaPipe)

Course: IPCV

Organisation:

Motilal Nehru National Institute of Technology Allahabad
Prayagraj – 211004, Uttar Pradesh, India
Department of Computer Science and Engineering

Submitted By — Team 10:

Student Name	Registration Number
Mukund Bihari Gupta	20233191
Sachin Patel	20233237
Sahil Singh	20233240
Sahil Raja	20233555

Academic Session: 2026–2027

Abstract

Drowsy driving is a severe road-safety hazard as fatigued drivers often fail to recognize declining alertness. This report presents a real-time, non-intrusive Driver Drowsiness Detection System built in Python using OpenCV, MediaPipe, Pygame, and Gmail SMTP. The application processes standard webcam feeds, tracking 468 3D facial landmarks and 21 hand landmarks. It calculates the Eye Aspect Ratio (EAR) for eyelid closure, Mouth Aspect Ratio (MAR) for yawning, and 3D head pose (yaw, pitch, roll) via OpenCV's solvePnP algorithm. Continuous eye closure (EAR < 0.23 for 5.0 s) or wide mouth opening (MAR > 0.60 for 1.5 s) triggers a 70% volume audio alarm (beep.mp3) and dispatches an automated emergency email with a webcam snapshot using an asynchronous daemon thread. Drivers can dismiss active alerts hands-free by holding a thumbs-up gesture across 5 consecutive frames. All events and session metrics are recorded in logs/driver_events.csv alongside an interactive Head-Up Display (HUD) overlay.

1. Introduction

1.1 Background

Operating a vehicle requires continuous cognitive and visual attention. Driver fatigue degrades reaction time and causes micro-sleeps. At 100 km/h, a vehicle travels ~139 metres in five seconds, making momentary lapses dangerous. Computer vision enables non-intrusive monitoring of physiological indicators like eye closure, yawning, and head slumping using a single dashboard camera without physical discomfort.

1.2 Problem Statement

Drivers often fail to assess fatigue onset objectively. There is a need for an affordable, real-time system that detects early behavioral symptoms, alerts the driver audibly, requires deliberate hands-free acknowledgment, and alerts emergency contacts with visual evidence.

1.3 Objectives

The core objectives of this project are:

1. To detect prolonged eye closure in real time by computing the bilateral Eye Aspect Ratio (EAR) from facial landmarks.
2. To identify driver yawning by calculating the Mouth Aspect Ratio (MAR) across inner lip and mouth corner landmarks.
3. To estimate 3D head pose angles (yaw, pitch, and roll) using a 6-point 3D facial model and OpenCV’s solvePnP solver.
4. To sound a continuous looping audio alarm upon detecting drowsiness and keep it active until the driver explicitly responds.
5. To implement a contactless thumbs-up hand gesture recognition mechanism (confirmed over 5 consecutive frames) for safe alarm dismissal.

6. To automatically capture a webcam snapshot of the drowsy driver and send an instant emergency email alert with the photo attachment using background multithreading.
7. To display a real-time monitoring dashboard overlay and record all session events and biometric metrics in a structured CSV log for post-trip analysis.

1.4 Scope

Designed for single-driver setups using a standard webcam (CAMERA_INDEX = 0). It uses a modular Python architecture (main.py for computer vision and logic; emergency.py for asynchronous email delivery). Its geometric pipeline is suitable for edge devices like Raspberry Pi or NVIDIA Jetson.

2. Methodology and System Architecture

2.1 System Overview

The system runs a frame-processing loop in main.py alongside emergency.py for alerts. On start, it prepares logging (logs/driver_events.csv), initializes Pygame audio (beep.mp3), and loads MediaPipe Face Mesh and Hands pipelines.

Each loop iteration captures and flips a frame, converts it to RGB, and extracts landmarks to compute EAR, MAR, and head pose. If thresholds are exceeded, the audio alarm triggers and emergency.py asynchronously emails a driver photo. An HUD overlay displays real-time metrics until the session ends via the 'Q' key.

2.2 Tools and Libraries

Table 1: Software Tools and Libraries Used

Library / Module	Role and Functionality in the System
Python 3	Core programming language used to build the vision, logic, and networking pipelines.
OpenCV (cv2)	Captures webcam video frames, flips/resizes images, draws eye polylines and HUD text, saves emergency JPG snapshots, and computes 3D head pose via solvePnP and Rodrigues.
MediaPipe (mediapipe)	Runs FaceMesh (1 face, refined landmarks, 0.5 detection/tracking confidence) and Hands (up to 2 hands, 0.6 detection/tracking confidence).
NumPy (numpy)	Performs fast Euclidean vector norm calculations (np.linalg.norm) and matrix operations for 3D-to-2D projection.

Library / Module	Role and Functionality in the System
Pygame (pygame.mixer)	Loads and plays the looping alert audio file (beep.mp3) at 70% volume without halting execution.
smtplib & email.message	Establishes an encrypted SSL connection (SMTP_SSL on port 465) to Gmail to transmit MIME-formatted emergency alert emails with JPEG attachments.
threading	Executes SMTP network transmission in a background daemon thread so webcam frame rates remain smooth and responsive.
csv, datetime, time, os	Manages filesystem directories, high-precision ISO timestamps, elapsed session timers, automatic photo cleanup, and CSV telemetry logging.

2.3 Eye Closure Detection (EAR)

MediaPipe Face Mesh extracts landmark indices for the right eye [33, 160, 158, 133, 153, 144] and left eye [362, 385, 387, 263, 373, 380]. Eye Aspect Ratio (EAR) is calculated as:

EAR = (||P2 - P6|| + ||P3 - P5||) / (2 * ||P1 - P4||)

Averaging both eyes provides stability. If EAR drops below EAR_THRESHOLD = 0.23 continuously for EYE_CLOSED_SECONDS = 5.0 seconds, prolonged closure is declared. Blinks reset the timer.

2.4 Yawn Detection (MAR)

Yawning is measured via Mouth Aspect Ratio (MAR) using upper inner lip (13), lower inner lip (14), left mouth corner (78), and right mouth corner (308):

MAR = ||Landmark 13 - Landmark 14|| / ||Landmark 78 - Landmark 308||

If MAR exceeds MAR_THRESHOLD = 0.60 for YAWN_SECONDS = 1.5 seconds, a yawn is detected. A 5-second cooldown prevents repeated logging.

2.5 3D Head Pose Estimation

6 key 2D facial landmarks—nose tip (1), chin (152), left eye corner (33), right eye corner (263), left mouth corner (61), right mouth corner (291)—are mapped to a 3D anatomical face model to derive pitch, yaw, and roll via cv2.solvePnP and cv2.Rodrigues.

2.6 Multi-Condition Alert and Hand Gesture Dismissal Logic

Drowsiness is triggered by: 1) Prolonged Eye Closure (EAR < 0.23 for 5.0 s), or 2) Prolonged Yawning (MAR > 0.60 for 1.5 s). Missing faces reset timers after 10 consecutive frames. Active alerts loop until dismissed via a Thumbs-Up gesture evaluated by MediaPipe Hands: thumb tip above joints, four fingers folded, and thumb separated horizontally. Detecting this

for GESTURE_CONFIRM_FRAMES = 5 consecutive frames stops the alarm and resets the alert state.

2.7 Automated Emergency Email Notification Module (emergency.py)

When an alert triggers, emergency.py captures a webcam snapshot to emergency_photos/, dispatches an email with the photo attachment via Gmail SMTP over SSL in a background thread, and deletes the local photo file immediately afterward in a finally block to ensure driver privacy.

2.8 Live Dashboard Overlay and CSV Event Logging

The HUD overlay displays session elapsed time, live EAR/MAR, counts for alerts/ yawns/ dismissals, status banners, and gesture hints. All state changes are appended to logs/driver_events.csv across eight metrics.

Table 2: System Configuration Parameters

Parameter Constant	Configured Value	Description
EAR_THRESHOLD	0.23	Eye Aspect Ratio threshold below which eyes are considered closed
EYE_CLOSED_SECONDS	5.0 seconds	Continuous eye closure duration required to trigger alarm
MAR_THRESHOLD	0.60	Mouth Aspect Ratio threshold above which mouth is considered wide open
YAWN_SECONDS	1.5 seconds	Continuous mouth opening duration required to classify a yawn
GESTURE_CONFIRM_FRAMES	5 frames	Consecutive thumbs-up frames required to dismiss active alarm
EMERGENCY_DELAY_SECONDS	60 seconds	Delay after alarm onset before capturing photo and sending emergency email
Face Missing Reset Limit	10 frames	Consecutive frames without a detected face before resetting timers
Alarm Audio Volume	0.7 (70%)	Playback volume for looping beep.mp3 alarm via Pygame

3. Results and System Behaviour

3.1 System Output and Operational Scenarios

Upon launch, main.py logs SESSION_START. Table 3 summarizes verified system behavior across operating scenarios:

Table 3: System Behaviour Across Operational Scenarios

Scenario / Input Condition	System Behaviour, Visual Output, and Telemetry Logged
Normal driving (eyes open, mouth closed)	Green eye contours drawn; banner shows MONITORING in green; live EAR and MAR update continuously.
Normal eye blink (< 0.5 seconds)	EAR briefly dips below 0.23; eye_closed_start resets immediately when eyes reopen; no false alarm.
Eyes closed continuously for >= 5.0 sec	Banner turns red (ALERT: PROLONGED EYE CLOSURE); beep.mp3 loops; alert counter increments; ALERT_TRIGGERED logged; snapshot saved and emailed via emergency.py; EMERGENCY_NOTIFICATION_TRIGGERED logged.
Mouth wide open for >= 1.5 sec	Yawn counter increments (5s cooldown); YAWN_DETECTED and ALERT_TRIGGERED (PROLONGED YAWNING) logged; alarm starts; emergency snapshot emailed.
Thumbs-up held for 5 consecutive frames	Hand skeleton drawn; alarm stops; status returns to MONITORING; dismissal counter increments; ALERT_DISMISSED logged; emergency trigger resets.
Driver face leaves camera view	Status turns red (NO FACE DETECTED); eye and yawn timers reset after 10 consecutive lost frames.
User presses 'Q' key	SESSION_END logged; camera, MediaPipe, and Pygame close cleanly; console prints full session summary.

3.2 Event Log Structure and Session Summary

1. State transitions are logged to logs/driver_events.csv (timestamp, session_elapsed_seconds, event, details, ear, mar, head_yaw, head_pitch). Tracked events include SESSION_START, YAWN_DETECTED, ALERT_TRIGGERED, EMERGENCY_NOTIFICATION_TRIGGERED, EMERGENCY_NOTIFICATION_ERROR, ALERT_DISMISSED, and SESSION_END. Console summaries are generated upon exit.

2. EMERGENCY_NOTIFICATION_TRIGGERED — Logged when the driver's snapshot is captured and background SMTP email delivery starts.

4. Discussion

4.1 System Strengths

- **Non-Intrusive & Multi-Cue Fusion:** Combines EAR, MAR, and head pose via standard webcam without extra hardware.
- **Asynchronous Alerts & Privacy:** Multithreaded SMTP prevents frame drops; immediate local image deletion preserves privacy.
- **Safe Gesture Dismissal:** 5-frame thumbs-up ensures conscious driver responsiveness without physical button interaction.

4.2 Practical Limitations

- **Static Thresholds & Lighting:** Fixed EAR/MAR values do not adapt to individual facial structures; standard webcams require adequate ambient lighting.
- **Speech Interference & Gesture Requirement:** Talking may elevate MAR momentarily; alarm requires manual gesture reset rather than auto-clearing upon eye opening.

4.3 Future Scope and Embedded Deployment Improvements

1. Driver Auto-Calibration: Implement a baseline calibration routine for personalized EAR/MAR thresholds.
2. Hardware & Sensor Upgrades: Integrate IR cameras for night monitoring and port to embedded edge devices (Raspberry Pi/NVIDIA Jetson) with GPS tracking.

5. Conclusion

This project delivered a Real-Time Driver's Drowsiness Detection System combining eye closure (EAR < 0.23), yawn detection (MAR > 0.60), audio alerts, multithreaded email notifications, thumbs-up gesture dismissal, and CSV event logging. The system provides a strong foundation for non-intrusive embedded vehicle safety deployments.